IDMC Milestone: Coding Section Playbook (60 Marks)

What this covers: your 4 coding questions: 2 Easy (10 marks each) and 2 Medium (20 marks each). It decodes what each one asks, gives a build plan with a verification checklist, and ends with drills and an exam-day routine.

> **How I built this:** the task wording comes from your Detailed Topic-wise Bifurcation screenshot. Parameter-file syntax, parameter types, taskflow steps and connection properties were checked against Informatica's current documentation. **I have not seen your lab, datasets or table names, and I don't know how marks are awarded.** Anything marked **[confirm]** is my inference. Check it in your lab or course notes before the exam.

0. The coding section at a glance

#	Level	Marks	Task (as written in your blueprint)	Skill cluster
C1	Easy	10	Create CDI connection for Flat File and create a Synchronisation Task for Insert and Update transaction	Connection + Sync Task
C2	Easy	10	Create CDI connection based on Parametrisation to load data into a Snowflake table	Parameterised connection + load
C3	Medium	20	S3 connection; mapping loading Snowflake with **connection parametrisation** for source and target; **parameter-file** driven loads using **Expression (Param)**, **Filter (Param)** and **Sort** before the target	Parameterised mapping + parameter file
C4	Medium	20	Load 2 CSV files into 2 target tables using a mapping with Filter + Expression; run the mapping tasks in a **parallel taskflow**; **e-mail on successful completion**	Mappings + mapping tasks + taskflow

Why this section decides your result: 60 of 100 marks are here, against 40 for the MCQs. Also, C3 and C4 together are worth 40 marks, as much as the entire MCQ section.

The pattern behind all four tasks

Every task follows the same chain, so one practised routine covers all of them:

```
Connection(s)  ->  Source/Target object  ->  Logic (sync task OR mapping)
     ->  Runnable task  ->  (Parameters / parameter file)  ->  (Taskflow +
notification)
     ->  RUN  ->  VERIFY in Monitor and in Snowflake
```

Task	How far down the chain it goes
C1	Connection, sync task, run
C2	Parameterised connection, load, run
C3	Everything up to the parameter file
C4	Everything up to the taskflow and the email

So C1 → C2 → C3 → C4 builds up. Practise them in that order.

---

1. Where things live in IDMC (decode the verbs)

The question says…	You do this	Where (typical)
"Create CDI connection"	Create a connection asset	**Administrator → Connections → New Connection**
"Synchronisation Task"	Run the sync task wizard	**Data Integration → New → Tasks → Synchronization Task**
"Create a Mapping"	Build source → transformations → target	**New → Mappings → Mapping**
"Mapping Task"	Make the mapping runnable	**New → Mapping Task** (pick the mapping)
"Parameter file"	Write a text file and point the mapping task to it	Mapping task → **Runtime Options**
"Taskflow"	Orchestrate tasks	**New → Taskflows**, then **Publish** it
"Verify"	Check run status, row counts, target data	**Monitor → My Jobs**, plus a Snowflake query

[confirm] Menu labels shift slightly between releases. The names above match current documentation, but rehearse once in your lab so you aren't hunting for buttons during the

exam.

---

2. Before-exam setup checklist (do this in the first 5 minutes)

☐	Check	Why
☐	The Secure Agent / runtime environment shows **Up and Running**	Flat-file connections and test connections need the agent
☐	You know the **runtime environment name** to select on every connection	Connections and tasks both ask for it
☐	You know the Snowflake **account, warehouse, database, schema, role, username**	Needed for the Snowflake connection
☐	You know the **S3 bucket / folder path, region, access key, secret key**	Needed for the S3 connection (C3)
☐	You know the **flat-file directory path** on the agent machine	Flat-file connection takes a directory, not a file
☐	You know the **project/folder name** to save assets in	The parameter file section heading depends on it
☐	You have a **naming plan** (see below)	Clean names help graders and help you

Naming convention (use it every time)

Asset	Pattern	Example
Connection	`CONN_<system>_<env>`	`CONN_FF_SRC`, `CONN_SF_TGT`, `CONN_S3_SRC`
Sync task	`SYNC_<source>_to_<target>_<op>`	`SYNC_Customers_to_SF_INS`
Mapping	`m_<source>_to_<target>`	`m_S3_to_SF_Orders`
Mapping task	`mt_<mapping name>`	`mt_m_S3_to_SF_Orders`
Taskflow	`tf_<purpose>`	`tf_Parallel_CSV_Load`

---

3. Building-block cheat sheet

3.1 Connections

Connection	Key properties	Watch out for
Flat File	Runtime environment, **Directory** (folder only, never the file name), Date Format (default `MM/dd/yyyy HH:mm:ss`), Code Page	You pick the **file name later in the task or mapping**. The directory must be reachable by the agent.
Snowflake Data Cloud	Runtime environment, Authentication = **Standard**, Username, Password, **Account**, **Warehouse**, **Role**, optional Additional JDBC URL parameters	The account name must not contain underscores. Use the alias if it does. Database/schema may be set in the connection or the JDBC parameters.
Amazon S3 V2 [confirm]	Runtime environment, Access Key, Secret Key, Folder Path / bucket, Region	Region and bucket must match the real bucket.

> **Always click Test Connection** and screenshot or note the success message.

3.2 Sync task wizard (C1)

Steps, in order: **Definition → Source → Target → Data Filters → Field Mapping → Schedule.**

Step	What you set
Definition	Task name, description, **Task Operation: Insert / Update / Upsert / Delete**
Source	Connection, source type (single object), source object, formatting options for flat files (delimiter, header)
Target	Connection, target object, (for Update/Upsert/Delete) the **key / update columns [confirm label]**
Data Filters	Leave **Process all rows** unless the question asks for a filter
Field Mapping	Map fields (**Automatch** helps), then **Validate Mapping**
Schedule	Run now (no schedule) unless asked. Email notification options also appear here.

3.3 Mapping parameters (C2, C3)

Input parameter types and where they work (from the docs):

Parameter type	Use it for	In which transformation
connection	Source, target or lookup connection	Source / Target / Lookup
data object	The source or target object (table or file)	Source / Target / Lookup
expression	A **full expression**	**Expression transformation**, Joiner condition, Lookup condition
field	A field name	Filter condition field, Joiner / Lookup condition fields
string	A value	Filter condition value
Filter, completely parameterized	The **whole** filter condition	**Filter transformation**
Sorter parameters	Sort field and sort order	**Sorter transformation**

Two tips straight from the documentation:

- **Build the mapping with real connections first.** Replace them with connection parameters at the end. Editing a source or target object while its connection is parameterised can cause runtime errors.
- To use a **filter on a parameterised source**, the filter itself must also be a parameter.

3.4 Parameter file rules (C3)

Rule	Detail
First line	`#USE_SECTIONS` if the file has sections. Without it only the first (global) section is read.
Sections	`[Global]`, and `[Project].[Folder].[TaskName]` or `[Project].[TaskName]` for one task
Syntax	`$$ParamName=value`, one parameter per line
Precedence	Task section > taskflow section > `#USE_SECTIONS` area > `[Global]`
Duplicates	In the same section the **first** value wins. The file is read **top-down**.
Missing parameter	If a parameter isn't in the file, the **value saved in the task** is used.
Referencing a parameter inside a value	Use a single `$`
Null values	A null for an overridable source/target object parameter makes the task **fail**
Connection parameters	The value is the **connection name** exactly as saved
Location	Default is a directory on the Secure Agent machine, and other locations can be chosen in the task's Runtime Options. **[confirm the default path in your lab]**

3.5 Taskflow steps (C4)

Step	Use
Data Task	Adds a mapping task or sync task to the taskflow and lets you set error handling and override parameters
Parallel Paths	Runs several items **at the same time**, then moves to the next step once the paths finish
Notification Task	Sends an **email** to the recipients you specify
Decision / Command / Wait / Throw / Subtaskflow	Not required for C4, but know what each does
Template to start from	**Parallel Tasks** (Start → Parallel Paths → End)

A taskflow must be **published** before it can run.

---

4. Task playbooks

C1 (Easy, 10 marks): Flat File connection + Sync Task for Insert and Update

Decoded: create a Flat File connection, then show **insert** and **update** behaviour with a Synchronization Task.

The ambiguity: "for Insert and Update Transaction" could mean (a) two tasks, one Insert and one Update, (b) one **Upsert** task, or (c) one task run twice. **The safest approach is to demonstrate both operations explicitly:**

Build plan

Step	Action	Done
1	Create the **Flat File connection** (runtime environment, directory, code page). **Test connection.**	☐
2	Confirm the **target** (likely a Snowflake table). If the question names a target connection, reuse it. If not, ask the invigilator or use the Snowflake connection from C2.	☐
3	**Task 1: Insert.** New → Tasks → Synchronization Task. Operation = **Insert**. Source = flat file. Target = table. Automatch fields, Validate Mapping, Save.	☐
4	**Run** Task 1. Check Monitor for success and rows processed.	☐
5	Change a few non-key values in the source file (or use the updated file provided).	☐
6	**Task 2: Update.** Copy or create a second task with Operation = **Update**. Choose the **key / update column(s)** (for example the ID). Run.	☐
7	**Verify in the target:** inserted row count is right, and the changed values appear after the update run.	☐
8	*(Optional)* Mention or build an **Upsert** variant to show you know the difference.	☐

What "good" looks like

- Connection test passes, task names are meaningful, field mapping is complete, both operations visibly worked.

Common mistakes

Mistake	Fix
Putting the **file name** in the connection directory	Directory only, pick the file in the task
Choosing **Update** with no key column	Choose the key / update column
Wrong delimiter or header setting	Check the source preview before mapping
Running Update before the rows exist	Run Insert first
Date format mismatch	Align the connection's Date Format with the file

C2 (Easy, 10 marks): Connection-parameterised load into a Snowflake table

Decoded: the **connection is a parameter**, not hardcoded. The same mapping works for any Snowflake connection chosen at run time.

Build plan

Step	Action	Done
1	Create the **Snowflake Data Cloud connection** (Standard authentication, account, warehouse, role, username, password). **Test.**	☐
2	Create (or reuse) the **source connection** (flat file or whatever the question gives).	☐
3	Build the **mapping**: Source → (optional Expression) → **Snowflake Target**. Use **real connections first**.	☐
4	In the **Target**, change the connection to **New Parameter** (type = connection, for example `$tgt_conn`). Do the same for the source if asked.	☐
5	Make sure the **target table exists**, or use the option to create it at runtime where available. **[confirm]**	☐
6	Save the mapping, then **New → Mapping Task**. Select the mapping. In the task, **choose the actual Snowflake connection** for the parameter.	☐
7	**Run.** Verify the job in Monitor and the rows in Snowflake.	☐

Verification query

```sql
SELECT COUNT(*) FROM <DB>.<SCHEMA>.<TARGET_TABLE>;
SELECT * FROM <DB>.<SCHEMA>.<TARGET_TABLE> LIMIT 10;
```

Common mistakes

Mistake	Fix
Parameterising the connection **before** selecting the object	Select the object with a real connection, then parameterise
Forgetting to **assign the connection in the mapping task**	The task asks for a value for every connection parameter
Table or schema case or name mismatch in Snowflake	Match the exact name and schema
Role without privileges	Use the role that owns or can write to the table

---

C3 (Medium, 20 marks): S3 → Snowflake with connection parameters, a parameter file, Expression (Param), Filter (Param) and Sort

Decoded sentence by sentence:

Phrase in the question	What you build
"Create CDI connection for AWS S3 bucket"	S3 connection
"mapping for loading data into Snowflake table"	S3 source → … → Snowflake target
"connection parametrisation for Source and Target"	Both connections are **connection parameters**
"data loads based on parameter-file configuration"	A **parameter file** supplies the values, and the **mapping task** points to it
"Expression (Param)"	An **Expression transformation** whose expression is an **expression parameter**
"Filter (Param)"	A **Filter transformation** whose condition is **parameterised**
"Sort before loading into Target"	A **Sorter** transformation placed before the target

Mapping layout

```
S3 Source (conn param)  ->  Filter (param)  ->  Expression (param)  ->  Sorter
->  Snowflake Target (conn param)
```

Build plan

Step	Action	Done
1	Create the **S3 connection** and the **Snowflake connection**. Test both.	☐
2	Create the mapping with **real connections**. Select the S3 file object and the Snowflake target table.	☐
3	Add **Filter**. Configure a normal condition first so you can validate, then change it to the **parameterised** condition (create the input parameter).	☐
4	Add **Expression**. Create a derived field and set its expression to an **expression-type input parameter**.	☐
5	Add **Sorter** on the required field(s), then link to the target.	☐
6	Convert **source and target connections** to **connection parameters**.	☐
7	Save and **validate** the mapping.	☐
8	Write the **parameter file** (template below) and save it where the task expects it.	☐
9	Create the **mapping task**. In **Runtime Options**, set the **parameter file name and location**. Make sure values not in the file are filled in the task.	☐
10	**Run.** Open the job in Monitor, check the logs for resolved parameter values, and confirm **row counts** (filtered rows are excluded).	☐
11	**Verify in Snowflake** that the rows are filtered, the derived column is populated and the order is sorted.	☐

Parameter file template

Adapt the project, folder, task and parameter names to your assets. Names are **case-sensitive**.

```
#USE_SECTIONS
[Global]
$$src_conn=CONN_S3_SRC
$$tgt_conn=CONN_SF_TGT

[Default].[Milestone].[mt_m_S3_to_SF_Orders]
$$filter_cond=STATUS = 'ACTIVE'
$$expr_total=QTY * UNIT_PRICE
```

Part	Meaning
`#USE_SECTIONS`	Required first line when the file has sections
`[Global]`	Applies to every task that uses these parameters
`[Default].[Milestone].[mt_...]`	Project `Default`, folder `Milestone`, task `mt_...`. Applies to that task only and **overrides Global**.
`$$name=value`	One parameter per line. Connection values are connection **names**.

[confirm] Check the exact value format for the filter and expression parameters (quoting, field names) by running once and reading the resolved values in the job log. If the task fails, simplify the value first, for example `STATUS='ACTIVE'`.

Common mistakes

Mistake	Fix
Missing `#USE_SECTIONS`	Sections are ignored and only the first global section is read
Section heading doesn't match project/folder/task	Copy names exactly from the Explore page
Parameter name differs (typo, `$` vs `$$`)	The file uses `$$name`
Same parameter defined twice in a section	The first one wins
Parameter file name or directory wrong	Match the file name exactly and put it where the agent can read it
Null value for an overridable source or target object parameter	The task fails, so always supply a value
Sorter after the target or on the wrong field	Place Sorter immediately before the target
Editing the object while the connection is parameterised	Reselect a real connection, edit, then parameterise again

---

C4 (Medium, 20 marks): 2 CSVs → 2 tables, parallel taskflow, email on success

Decoded:

Phrase	What you build
"Load data from 2 CSV files to 2 different target tables"	**Two independent pipelines** (CSV A → Table A, CSV B → Table B)
"mapping with Filter and Expression"	Each mapping: Source → **Filter** → **Expression** → Target
"use the mappings inside Mapping Tasks"	**Two mapping tasks**, one per mapping
"parallel Taskflow"	A taskflow with a **Parallel Paths** step containing both **Data Tasks**
"trigger an e-mail on successful completion of data load"	A **Notification Task** step **after** the Parallel Paths step

Design

```
Start
  |
Parallel Paths
  |-- Data Task: mt_m_CSV1_to_T1
  |-- Data Task: mt_m_CSV2_to_T2
  |
Notification Task (email)   <- runs only after BOTH paths finish
  |
End
```

Build plan

Step	Action	Done
1	Create the **flat-file connection** (for the CSVs) and the **Snowflake connection** (for the targets). Test both.	☐
2	**Mapping 1:** CSV1 → Filter → Expression → Table1. Validate and save.	☐
3	**Mapping 2:** CSV2 → Filter → Expression → Table2. Validate and save. (Copy mapping 1, then change source and target.)	☐
4	Create **mapping task 1** and **mapping task 2**. Run each **once on its own** to confirm both work before building the taskflow.	☐
5	**New → Taskflows.** Start from the **Parallel Tasks** template.	☐
6	Put a **Data Task** in each path of **Parallel Paths** and select mapping task 1 and 2.	☐
7	After the Parallel Paths step, add a **Notification Task**. Set recipients, subject and body (for example the success message).	☐
8	**Save, validate and Publish** the taskflow.	☐
9	**Run** the taskflow. In Monitor, confirm both data tasks ran **concurrently** and the notification step ran **last**.	☐
10	Confirm the **email arrived** and check **both tables** in Snowflake.	☐

Common mistakes

Mistake	Fix
Placing the Notification Task **inside** one parallel path	Place it after the Parallel Paths step so it waits for both
Building the taskflow before testing the tasks	Test each mapping task alone first
Forgetting to publish	Unpublished taskflows can't run
Mapping both CSVs into one pipeline	The question asks for 2 targets, so keep 2 independent flows
Target tables missing	Create them in Snowflake or use create-at-runtime where available **[confirm]**
Email recipient left blank or wrong	Use the address you can access
Filter or Expression not meaningful	Use real conditions, for example `STATUS='ACTIVE'`, and a real derived field

5. Time and marks strategy

Order	Task	Marks	Suggested share of coding time
1	C1	10	about 15%
2	C2	10	about 15%
3	C3	20	about 35%
4	C4	20	about 35%

[confirm] I don't know your exam duration, so these are proportions of your total coding time.

Rules of thumb:

- **Create all connections first.** C1 to C4 reuse them.
- **Test before you build further** (test connection, run the task alone, then the taskflow).
- **A partial, running solution beats a perfect unfinished one.** Get a plain working version first, then add the parameters.
- **Save often.** Screenshot or note the success of every connection test and run.

- If stuck on C3's parameter file, simplify: first hardcode the filter and expression, run, and only then parameterise.

---

6. Troubleshooting cheat sheet

Symptom	Likely cause	Fix
Connection test fails	Agent not running, wrong runtime environment, wrong credentials	Check the agent status, runtime environment and credentials
Snowflake connection fails	Account name format, underscores, wrong role or warehouse	Use the correct account identifier or alias
Flat file not visible	Directory is wrong or the file isn't in that directory	Directory is a folder path, and the file must exist in it or a subfolder
Parameter not applied	Missing `#USE_SECTIONS`, wrong heading, typo	Re-check the file structure and spelling
Task fails on a null parameter	Overridable object parameter has no value	Provide a value in the file or task
Update doesn't change rows	No key or update column, key mismatch	Set the key column and confirm values match the target
Rows missing in target	Filter too strict, field mapping gaps	Check the filter value and the job log row counts
Output not sorted	Sorter placed before a step that reorders, or wrong key	Put Sorter last before the target
Taskflow won't run	Not published	Publish it
No email	Notification step missing, wrong placement or recipient	Check the step, position and address

---

7. Practice drills (rehearse until each takes under 15 minutes)

Drill	Based on	Variation to practise
D1	C1	Insert task, then an **Upsert** task on the same file
D2	C1	Update task with a **composite key**
D3	C2	Same mapping, run with **two different Snowflake connections**
D4	C2	Parameterise the **target object** as well as the connection
D5	C3	Move all parameter values to the file, then **change one value** and re-run
D6	C3	Add a **second task section** in the same file with different values
D7	C4	Add a **third parallel path**
D8	C4	Add a **Decision** after the parallel step, with an email on failure too
D9	C4	Put the **parameter file** values into the taskflow run
D10	All	A full **timed mock** of C1 to C4 in one sitting

---

8. Exam-day routine

1. Confirm the agent is **Up and Running**.
2. Create **all connections** and test them.
3. Do **C1 → C2 → C3 → C4**. After each, **run and verify**.
4. Before leaving a task, check: connection tested ✓, validated ✓, **run succeeded ✓**, **data verified ✓**, names clear ✓.
5. In the last minutes, re-check C4: **taskflow published**, **email step placement**, **both tables loaded**.

Final self-check before submitting

☐	Item
☐	Every connection tested successfully
☐	C1: both Insert and Update demonstrated and verified
☐	C2: connection is a parameter and is assigned in the mapping task
☐	C3: parameter file correct, task points to it, Filter/Expression/Sort behave as expected
☐	C4: two mapping tasks, parallel paths, notification after both, taskflow published, email received
☐	Row counts match expectations in Snowflake

9. What I verified vs what you should confirm

Item	Status
Parameter file: `#USE_SECTIONS`, `[Global]`, `[project].[folder].[task]`, `$$param=value`, precedence, first-value-wins, top-down	**Verified** in Informatica documentation
Input parameter types and which transformations accept them (connection, data object, expression, field, string, Filter, Sorter)	**Verified**
Tip to build with real connections first, then parameterise	**Verified**
Sync task wizard steps and Insert / Update / Upsert / Delete operations	**Verified** (documented across connector guides)
Flat file connection properties (directory, date format, code page)	**Verified**
Snowflake Data Cloud connection fields (Standard auth, account, warehouse, role)	**Verified**
Taskflow steps (Data Task, Parallel Paths, Notification Task) and the Parallel Tasks template	**Verified**
S3 V2 connection field names, sync-task key column label, default parameter-file path, "create target at runtime"	**[confirm in lab]**
Exact value format for Filter and Expression parameters in the parameter file	**[confirm by running once]**
How the exam awards marks, its duration, and your dataset and table names	**Unknown to me**: confirm with your instructor or exam brief

---

Next step

If you can share your **lab details** (connection names, sample file columns, target table structure) and anything your instructor said about **marking**, I can turn these playbooks into exact, copy-ready steps and mock datasets for each task.